

Elaborato per il corso di Virtualizzazione e integrazione di Sistemi A.A 2025/2026

Progettazione ed Implementazione di una Architettura Web Zero-Trust

Studente: Todeschi Matteo (matteo.todeschi2@studio.unibo.it)
Matricola: 0001117493

Indice

Capitolo 1: Obiettivi di Progetto	3
1.1 Il contesto dei microservizi moderni e le sfide di sicurezza	3
1.2 Obiettivi chiave: Identity-Aware API Gateway e Mesh.....	4
1.3 Presentazione dell'applicazione didattica.....	5
Capitolo 2: Scelte Progettuali e Architettura	6
2.1 Il paradigma di sicurezza Zero-Trust.....	6
2.2 Segmentazione della rete a 3 livelli.....	6
2.3 Il pattern Forward-Auth al perimetro	7
2.4 Disaccoppiamento dei microservizi di backend.....	8
Capitolo 3: Tecnologie, Protocolli e Glossario	10
3.1 Server SSO ed OpenID Connect	10
3.2 Il Pattern Sidecar Proxy e l'architettura di Envoy.....	11
3.3 Mutua Autenticazione (mTLS) e crittografia est-ovest.....	11
3.4 Dynamic Routing con Traefik.....	12
Capitolo 4: Verifica e Collaudo del Sistema.....	13
4.1 Collaudo della Sicurezza Perimetrale	13
4.2 Collaudo del Service Mesh mTLS e Admin Port di Envoy.....	14
4.3 Collaudo del Flusso Utente End-to-End	15
Capitolo 5: Trasparenza sull'Uso dell'AI Generativa	16
Capitolo 6: Aspetti Implementativi Rilevanti.....	17
6.1 Bootstrapping dinamico dei secret in avvio	17
6.2 Gestione del Single Log-Out (SLO)	17
6.3 Gestione della compatibilità Dual-Environment.....	17
Capitolo 7: Ambiente di produzione.....	18
Capitolo 8: Schema complessivo del sistema	19

Capitolo 1: Obiettivi di Progetto

1.1 Il contesto dei microservizi moderni e le sfide di sicurezza

Per decenni, la sicurezza delle architetture informatiche si è basata su un modello di tipo perimetrale noto come approccio **Castle-and-Moat**. In questo scenario, la rete locale viene trattata come una fortezza dove il perimetro esterno agisce come fossato (Moat) con lo scopo di proteggere il castello (Castle) dalle minacce esterne.

Uno dei presupposti cardine di questo approccio è la **fiducia implicita**, ovvero la consapevolezza che una volta ottenuto il lasciapassare tramite l'autenticazione iniziale, qualsiasi dispositivo all'interno della rete viene considerato automaticamente fidato.

Questo aspetto porta ad un **Single Point of Failure** in quanto, dando per assodato che l'interno della rete sia sempre un ambiente sicuro, concentra la sicurezza solo nella fase di autenticazione (il fossato) permettendo quindi ad un malintenzionato di ottenere accesso illimitato alle risorse interne al sistema dopo aver violato il perimetro esterno.

Questa problematica, unita al bisogno di adattarsi alle nuove tecnologie di orchestrazione, ha portato allo sviluppo di un nuovo pattern architetturale denominato **Zero-Trust**, fondato sul principio guida:

"Never Trust, Always Verify".

In questo modello innanzitutto non esiste nessun confine implicito, infatti la posizione di un utente o di un microservizio non garantisce alcun diritto di accesso ad altre risorse del sistema.

Inoltre ogni singola richiesta di comunicazione deve essere esplicitamente autenticata, autorizzata e cifrata prima di essere processata.

Tutto ciò permette quindi di rispettare il **Principio del minimo** privilegio, secondo cui ogni componente ha accesso esclusivamente alle risorse strettamente necessarie per svolgere la propria funzione.

1.2 Obiettivi chiave: Identity-Aware API Gateway e Mesh

Per rispondere concretamente alle sfide di sicurezza e di gestione sollevate dal modello Zero-Trust, il lavoro svolto si articola su due pilastri tecnologici fondamentali con lo scopo di garantire un'elevata sicurezza del traffico di rete.

1) Identity-Aware Perimetrale

Il primo obiettivo riguarda il controllo del traffico in ingresso dall'esterno verso l'interno dell'infrastruttura (traffico Nord-Sud).

L'approccio tradizionale delega spesso il controllo degli accessi ai singoli servizi; in questa architettura, invece, la gestione dell'identità viene centralizzata sul perimetro:

- **Autenticazione Centralizzata:** Integrazione dei protocolli **OpenID Connect (OIDC)** e **Single Sign-On (SSO)** per gestire le credenziali e le sessioni utente in modo unificato.
- **Validazione all'API Gateway:** L'API Gateway agisce come unico punto di ingresso e filtro *identity-aware*. Nessuna richiesta generata dall'esterno può raggiungere i microservizi sottostanti senza che l'identità dell'utente sia stata preventivamente verificata e convalidata.

2) Service Mesh e mTLS

Una volta superato il perimetro esterno, applicare il modello Zero-Trust significa non fidarsi della rete privata interna.

L'obiettivo è mettere in sicurezza il traffico interno (Est-Ovest):

- **Isolamento dei Servizi:** Innanzitutto si applica la separazione delle reti virtuali per impedire comunicazioni dirette non autorizzate ed il binding del backend su 127.0.0.1, rendendo quindi i microservizi inaccessibili dall'esterno.
- **Cifratura Bidirezionale (mTLS):** Viene poi implementata una **Service Mesh** basata sui sidecar proxy **Envoy**. Tutte le comunicazioni interne sono cifrate e autenticate mutuamente tramite il protocollo **Mutual TLS**. In questo modo, ogni container verifica l'identità del servizio chiamante tramite certificati digitali gestiti in modo dinamico e trasparente rispetto al codice applicativo.

1.3 Presentazione dell'applicazione didattica.

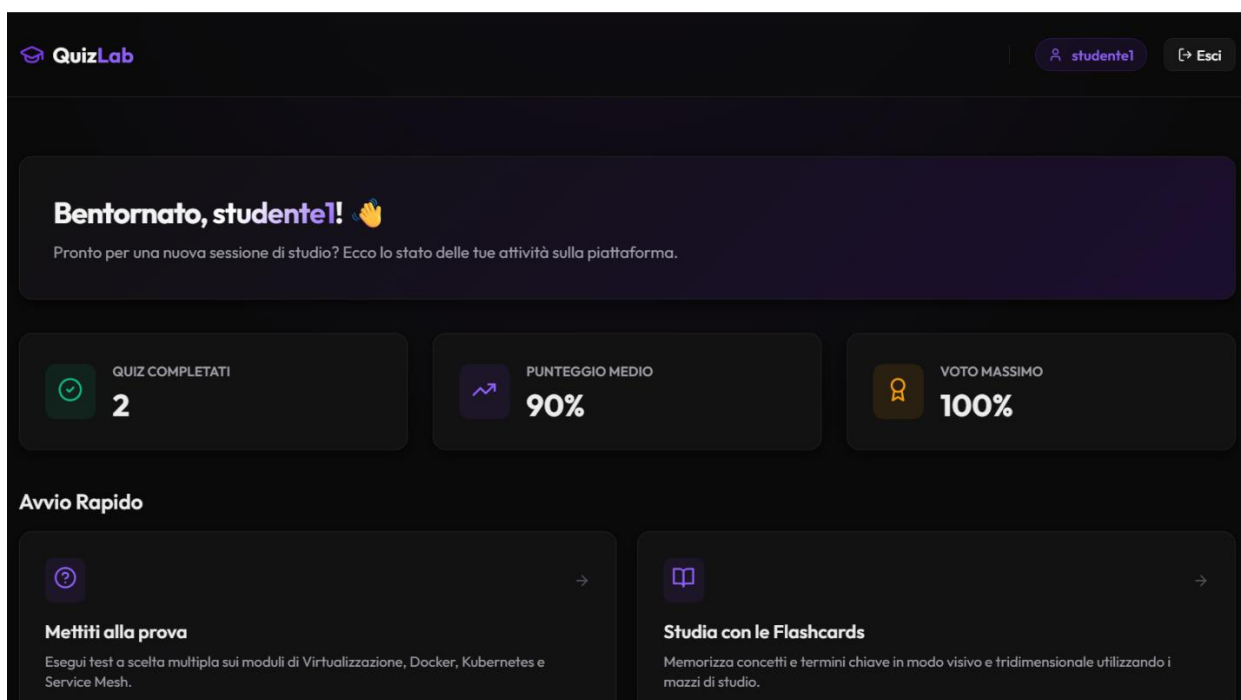
L'applicativo che ho deciso di realizzare per mettere in pratica questo approccio implementativo prende il nome di “**Quizlab**”.

Si tratta di una piattaforma digitale di aiuto all'apprendimento basata sulla fruizione di contenuti di autovalutazione.

L'applicazione web è stata realizzata usando il framework React in ottica di fornire una Single Page Application ed è suddivisa in 4 moduli principali:

- **Modulo di Autenticazione:** Schermata pubblica di benvenuto che consente agli studenti di registrarsi o accedere.
- **Dashboard con Storico Punteggi:** Pannello di controllo che mostra i progressi dello studente ed i risultati dei test precedenti.
- **Sezione Quiz:** Permette di svolgere i test ufficiali o di **creare quiz personalizzati** con domande a risposta multipla, spiegazioni didattiche ed immagini esplicative.
- **Sezione Flashcard:** Workspace dedicato alla creazione di mazzi di flashcard con interfaccia a scorrimento (fronte/retro).

Di seguito una cattura della Dashboard principale dell'applicazione:



Capitolo 2: Scelte Progettuali e Architettura

2.1 Il paradigma di sicurezza Zero-Trust

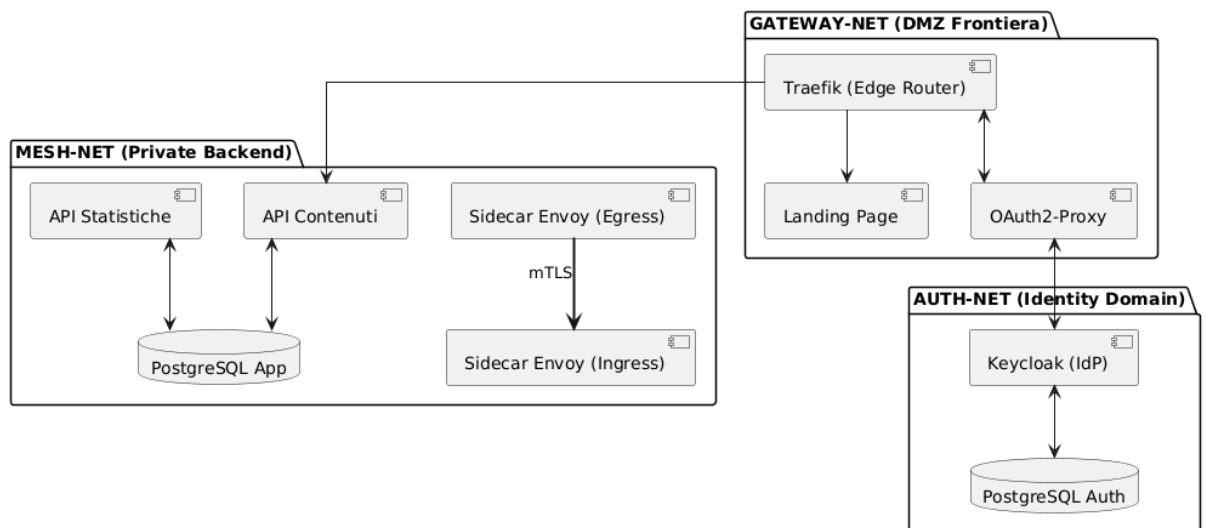
Partendo dalle fondamenta del paradigma Zero-Trust, il progetto si basa su 3 principi fondamentali:

- L'origine del pacchetto non garantisce sicurezza → l'IP sorgente non viene mai usato per autenticare un utente o un servizio interno, infatti l'autenticazione richiede sempre un token OIDC verificato o un certificato digitale mTLS.
- Controllo della superficie d'attacco → Il backend non espone porte pubbliche sull'host e risponde solo sull'interfaccia di loopback (127.0.0.1).
- Isolamento del traffico interno: Il traffico tra microservizi non viaggia mai in chiaro sulla rete privata, eliminando il rischio di intercettazione interna.

2.2 Segmentazione della rete a 3 livelli

Per garantire l'isolamento dei sottocomponenti, lo stack di Docker Compose non usa un'unica rete di default, bensì 3 reti bridge virtuali isolate, impedendo quindi la comunicazione diretta tra componenti con ruoli diversi o non autorizzati.

- **gateway-net:**
Essa è la rete perimetrale pubblica che contiene l'Ingress Gateway (Traefik), il middleware di sicurezza (OAuth2-Proxy) e la Landing Page (DMZ).
È l'unica rete ad avere il permesso di comunicare con l'esterno.
- **auth-net:**
È la rete che si occupa della gestione dell'identità e contiene Keycloak, il database Postgres-Auth ed un container di setup.
- **mesh-net:**
Questa rete, priva di accesso a internet, è dedicata al backend e contiene i microservizi con i relativi sidecar Envoy ed il database applicativo (postgres-app).



2.3 Il pattern Forward-Auth al perimetro

Per gestire l'accesso sicuro alle risorse senza appesantire i microservizi interni, l'architettura adotta il pattern Forward-Auth, basato sul principio fondamentale della Separazione delle Responsabilità dividendo il ruolo di instradamento del traffico da quello di verifica dell'identità.

L'API Gateway si occupa esclusivamente del routing di rete e della gestione del traffico in ingresso senza dover conoscere le logiche interne di cifratura OAuth2 o la gestione dei token.

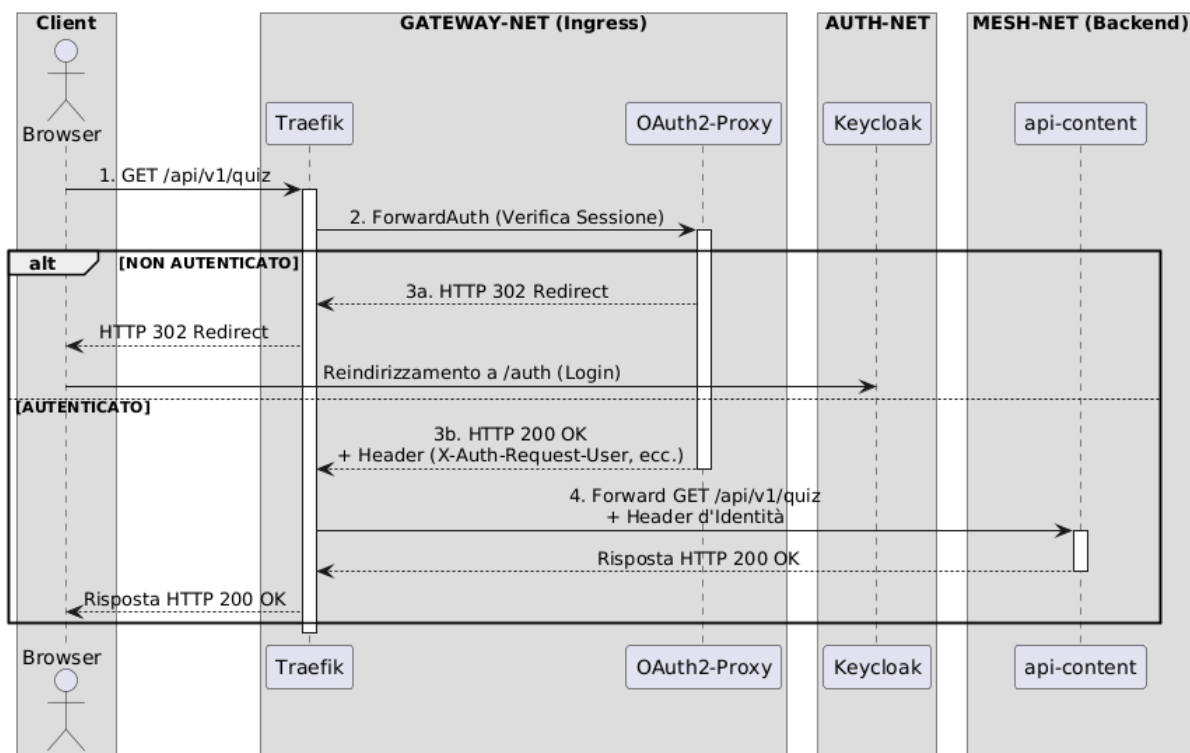
La verifica delle credenziali e dello stato della sessione infatti è delegata interamente a componenti dedicati.

In particolare Keycloak agisce come Identity Provider OIDC, mentre OAuth2-Proxy valida le sessioni e garantisce che solo le richieste autenticate possano proseguire.

Il Flusso di Autenticazione:

Il client invia una richiesta per accedere a una risorsa protetta, Traefik intercetta la richiesta in ingresso e, prima di inoltrarla al backend, invoca il middleware di autenticazione che si occuperà di verificare lo stato dell'utente.

Se la sessione è valida la richiesta viene inoltrata al backend che fornirà i dati necessari per la risposta, in caso contrario l'utente viene inoltrato alla schermata di login per completare l'autenticazione.



Il principale beneficio di questo modello risiede nella decentralizzazione della sicurezza dal codice applicativo: i microservizi non hanno il compito di gestire la cifratura dei token o le sessioni degli utenti. Il backend deve semplicemente fidarsi degli header d'identità iniettati.

2.4 Disaccoppiamento dei microservizi di backend

Per garantire modularità l'applicazione è stata pensata per garantire la separazione dei vari domini applicativi.

In particolare è presente il backend principale denominato **api-content** che è responsabile delle operazioni di base e gestisce le operazioni di CRUD (Create, Read, Update, Delete) per quiz e flashcard, interfacciandosi con il database postgres-app.

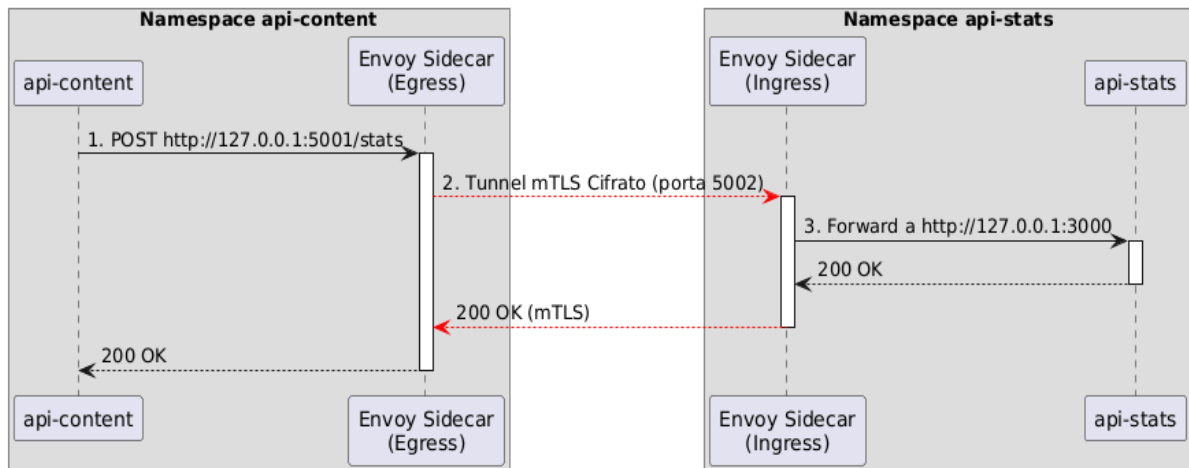
Esiste poi un secondo servizio denominato **api-stats** completamente isolato, adibito unicamente alla registrazione e all'elaborazione dei punteggi e dello storico utente.

A questo punto risulta fondamentale capire la maniera in cui i due microservizi si interfacciano tra di loro.

Quando un utente completa e sottomette un quiz, il servizio api-content deve notificare il punteggio ottenuto ad api-stats.

In un'architettura tradizionale, questa chiamata avverrebbe direttamente via HTTP in chiaro sulla rete locale.

Secondo i principi Zero-Trust invece, la comunicazione tra i vari servizi (traffico Est-Ovest) viene incapsulata e cifrata attraverso la Rete Mesh.



In particolare api-content non invia la richiesta HTTP direttamente alla rete, ma la dirotta sul proprio proxy Envoy Sidecar in ascolto sull'interfaccia locale.

Il proxy Envoy Egress autentica e cifra la chiamata tramite il protocollo mTLS, trasmettendola attraverso la rete mesh-net verso il proxy Ingress di api-stats.

Il proxy Envoy di destinazione verifica il certificato client del chiamante, decifra il traffico e lo consegna al microservizio api-stats, il quale ascolta esclusivamente sull'interfaccia di loopback locale.

Questo approccio garantisce la totale separazione dei domini applicativi. Poiché il servizio api-stats risponde solo su localhost ed è raggiungibile unicamente dal proprio proxy Envoy tramite certificati mTLS validi, le statistiche e i punteggi non possono in alcun modo essere alterati o invocati direttamente dall'esterno, né da utenti malintenzionati né da altri container non autorizzati all'interno della rete.

Capitolo 3: Tecnologie, Protocolli e Glossario

3.1 Server SSO ed OpenID Connect

Keycloak

Per la gestione centralizzata delle identità, degli accessi e dei meccanismi di Single Sign-On (SSO), è stato adottato Keycloak ed in particolare la versione 24, basata sull'architettura Quarkus (un framework Java *cloud-native*) che porta a vari miglioramenti rispetto alle versioni precedenti come ad esempio la riduzione di uso della memoria e la riduzione dei tempi di boot dei container.

Il Protocollo OpenID Connect (OIDC)

L'autenticazione degli utenti si basa sul protocollo OIDC, un Identity Layer sviluppato al di sopra del framework di autorizzazione **OAuth 2.0**. In aggiunta ad OAuth 2.0, che già permette di gestire l'autorizzazione all'accesso delle risorse, OIDC consente anche ai client di verificare l'identità dell'utente sulla base dell'autenticazione eseguita dall'Identity Provider, nel nostro caso Keycloak.

Il protocollo rilascia due tipologie di token principali, rappresentati sotto forma di **JSON Web Token (JWT)** firmati crittograficamente:

- **ID Token:** Contiene i dati e gli attributi dell'utente
- **Access Token:** L'autorizzazione necessaria ad accedere alle API e ai servizi protetti.

Single Log-Out (SLO)

In un'architettura distribuita basata su OIDC, la semplice eliminazione dei cookie di sessione nel browser locale non è sufficiente a garantire il completo *logout*, infatti se la sessione globale sull'Identity Provider rimane attiva, l'utente potrebbe accedere nuovamente dallo stesso device senza dover reinserire le credenziali.

Per ovviare a questo problema è stato utilizzato il **Single Log-Out** che consiste nel permettere a OAuth2-Proxy di inviare una notifica diretta a Keycloak al momento della disconnessione che provvederà quindi a invalidare la sessione SSO globale a livello centralizzato, distruggendo il token.

3.2 Il Pattern Sidecar Proxy e l'architettura di Envoy

Il **Sidecar Pattern** è un modello architetturale secondo cui nello stesso stack di rete del container applicativo principale viene posizionato un container ausiliario.

Il vantaggio di questo pattern è il Networking Offloading, ovvero il fatto che tutte le funzionalità avanzate di gestione della rete vengono delegate interamente al proxy sidecar facendo quindi in modo che i servizi di backend rimangano completamente all'oscuro rispetto alle logiche di cifratura di rete, riducendo le dimensioni del codice applicativo e la superficie di attacco.

Come motore della Service Mesh è stato scelto **Envoy Proxy**, un proxy sviluppato in C++.

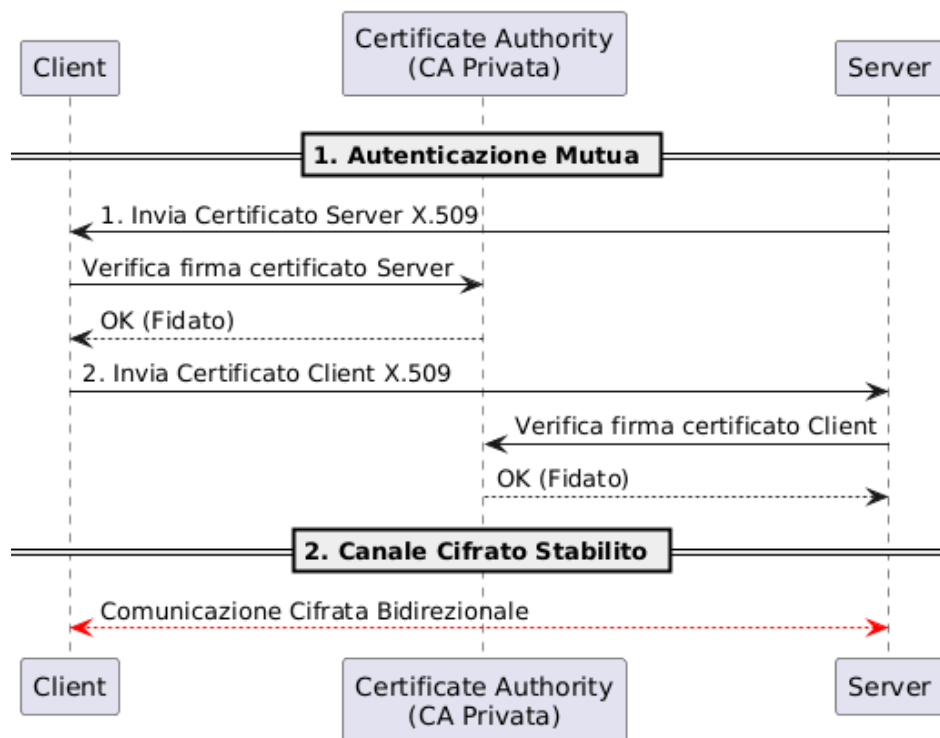
Envoy opera a Livello applicativo (L7) dello stack OSI ed è in grado di decodificare e processare il traffico di protocolli come TLS e HTTP con le loro varie estensioni, consentendo di applicare controlli di sicurezza basati sulle intestazioni dei messaggi HTTP.

3.3 Mutua Autenticazione (mTLS) e crittografia est-ovest.

TLS (Transport Layer Security) è un protocollo di sicurezza standard utilizzato su Internet per cifrare la comunicazione tra due dispositivi, ovvero è ciò che trasforma una connessione visibile a chiunque in un canale privato e protetto.

Esistono due versioni applicative del protocollo TLS:

- **TLS Standard (HTTPS):** Il server presenta un certificato digitale per dimostrare la propria identità al client, garantendo che i dati scambiati siano cifrati. Il client rimane però anonimo a livello di trasporto.
- **Mutual TLS (mTLS):** Applicato al traffico interno tra microservizi impone che **entrambi gli attori** della comunicazione presentino ciascuno un proprio certificato digitale **X.509** valido, emesso e firmato da una **Certificate Authority (CA)** comune e fidata.



La sicurezza del traffico interno alla rete mesh si basa sui seguenti punti:

- Certificate Authority (CA): Una CA auto-firmata, nel progetto generata tramite lo script `generate-certs.sh`
- I certificati dei sidecar vincolano la propria validità solo ad hostname specifici.
- La verifica mTLS è un vincolo bloccante durante l'handshake TCP. Qualsiasi connessione priva di un certificato client valido firmato dalla CA viene scartata all'istante.

3.4 Dynamic Routing con Traefik

Per concludere il cerchio, è giusto far notare che il controllo del traffico in ingresso è affidato a Traefik, un Edge Router.

A differenza dei reverse proxy tradizionali, Traefik è pensato per interfacciarsi direttamente con il demone Docker (`/var/run/docker.sock`), effettuando Service Discovery dinamica: rileva in tempo reale l'avvio o l'arresto dei container senza richiedere riavvii di servizio.

Tramite l'aggancio alla configurazione dinamica fornita, Traefik applica il middleware Forward-Auth: per ogni richiesta diretta alle rotte protette, interpella il proxy di autenticazione oauth2 e prosegue con l'inoltro al backend solo a seguito di un esito positivo con risposta 200.

Capitolo 4: Verifica e Collaudo del Sistema

La fase di collaudo ha l'obiettivo di dimostrare l'efficacia delle misure di sicurezza implementate, la robustezza del Service Mesh e la corretta integrazione dei servizi nel flusso utente *end-to-end*.

4.1 Collaudo della Sicurezza Perimetrale

L'obiettivo è verificare che l'architettura impedisca qualsiasi accesso non autorizzato alle API di backend.

Per fare ciò è stata simulata una chiamata diretta HTTP tramite strumento curl verso gli endpoint privati del backend.

```
$ curl -i https://quizlab.techmatrix.it/api/v1/quiz

HTTP/1.1 302 Found
Location: https://quizlab.techmatrix.it/auth/realms/QuizLab/protocol/openid-connect/auth?...
Set-Cookie: _oauth2_proxy_csrf=...; Path=/; Secure; HttpOnly
Server: cloudflare
Via: 1.1 Caddy
```

Dall'esito risulta evidente come l'API Gateway risponda con un codice 302 Found, impedendo la fuga di dati dal backend e reindirizzando obbligatoriamente il client verso il server SSO Keycloak.

Si è provato allora a simulare un tentativo di intrusione in cui un ipotetico attaccante, avendo compromesso il container della Landing Page nella DMZ, cerca di interrogare direttamente il backend api-content:3000 sulla rete interna senza passare dall'API Gateway:

```
root@mesh-service:/opt/quizlab:
docker compose exec landing-page curl -i http://api-content:3000/api/v1/quiz

HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Content-Type: application/json; charset=utf-8
Content-Length: 173
ETag: W/"ad-2bvG8U7DhAYH0Kh/FIXgEu06rFw"
Date: Tue, 21 Jul 2026 14:50:43 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"error":"Unauthorized","message":"Accesso diretto vietato. Questa risorsa è protetta e accessibile esclusivamente tramite l'API Ingress Gateway con sessione OIDC attiva."}
```

Questo esito dimostra che nell'architettura anche in caso di compromissione di un container perimetrale, i microservizi di backend rifiutano autonomamente qualsiasi chiamata diretta non autenticata.

4.2 Collaudo del Service Mesh mTLS e Admin Port di Envoy

L'obiettivo è convalidare la riservatezza e l'integrità delle comunicazioni inter-servizio tra api-content ed api-stats, dimostrando sia l'isolamento di rete dei container, sia l'effettiva cifratura mTLS gestita dai sidecar.

A) Verifica dell'isolamento del backend api-stats

- **Idea:** Dimostrare che il microservizio api-stats è completamente sordo alle chiamate di rete dirette provenienti da altri container e risponde esclusivamente sull'interfaccia di loopback (127.0.0.1).
- **Esito:** La chiamata fallisce immediatamente restituendo l'errore "fetch failed", confermando che l'applicazione non espone la porta 3000 sulla rete condivisa.

B) Verifica dell'instradamento corretto tramite Sidecar mTLS

- **Idea:** Verificare che la chiamata inviata dal backend api-content al proprio sidecar Envoy locale venga correttamente incapsulata in un tunnel mTLS e consegnata con successo ad api-stats.
- **Esito:** Si ottiene una risposta HTTP con stato HTTP/1.1 200 OK contenente i dati JSON restituiti dal microservizio delle statistiche.

C) Ispezione delle metriche mTLS tramite le Admin Port di Envoy

- **Idea:** Consultare la console di amministrazione di Envoy per confermare l'avvenuta negoziazione dei certificati mTLS.
- **Esito:** L'output conferma l'avvenuta cifratura mostrando i contatori degli handshake SSL/TLS andati a buon fine.

```
root@mesh-service:/opt/quizlab

docker compose exec api-content node -e "fetch('http://api-stats:3000/api/v1/stats').then(r => console.log('STATUS:', r.status)).catch(e => console.log('ESITO:', e.message))"

ESITO: fetch failed

-----

root@mesh-service:/opt/quizlab

docker compose exec api-content node -e "fetch('http://127.0.0.1:5001/api/v1/stats/studente1').then(r => console.log('STATUS:', r.status)).catch(e => console.log('ESITO:', e.message))"

STATUS: 200

-----

root@mesh-service:/opt/quizlab

docker compose exec api-content node -e "fetch('http://127.0.0.1:9901/stats').then(r => r.text()).then(t => console.log(t.split('\n').filter(l => l.includes('ssl.handshake'))))"

[ 'cluster.stats_service_cluster.ssl.handshake: 13' ]
```

Sniffing del traffico cifrato con tcpdump

- **Idea:** Dimostrare strumentalmente che il traffico in transito sulla porta mTLS 5002 è del tutto cifrato e protetto.
- **Esito:** Con il comando tcpdump in ascolto, è stato svolto un quiz. Dall'analisi dei dump esadecimali ed ASCII si evince che nessun dato applicativo viaggia in chiaro.

```
root@mesh-service:/opt/quizlab

tcpdump -i any port 5002 -c 5 -X

15:15:53 IP 172.22.0.4.50336 > 172.22.0.3.5002: Flags [S], length 0
    0x0000: 4500 003c dfe3 4000 4006 02a5 ac16 0004  E..<..@.....
    0x0010: ac16 0003 c4a0 138a 743b 6d2e 0000 0000  ....t;m.....
    0x0020: a002 faf0 5862 0000 0204 05b4 0402 080a  ....Xb.....

5 packets captured
```

4.3 Collaudo del Flusso Utente End-to-End

Fase 1 (Login OIDC):

- Collegandosi da un nuovo device e digitando il dominio dell'applicazione si viene subito indirizzati alla landing page che invita ad eseguire il login SSO tramite Keycloak.
- Inserendo le credenziali corrette si viene portati nella vera Dashboard dell'applicativo

Fase 2 (Operazioni CRUD):

- È stato verificato il corretto funzionamento dell'aggiunta di quiz alla piattaforma con domande multiple e immagini.
- Aprendo i DevTools del Browser inoltre compaiono evidenti richieste di tipo POST che ne dimostrano il funzionamento.

Fase 3 (Esecuzione Quiz e Salvataggio):

- Dopo aver terminato un quiz compare correttamente un messaggio di avvenuto salvataggio oltre che la relativa risposta 200 nel pannello Network dei DevTools.

Fase 4 (Logout):

- Cliccando su “Esci”, l'utente viene riportato sulla landing page ed è obbligato a rieseguire il Login Keycloak per accedere nuovamente alla piattaforma

Capitolo 5: Trasparenza sull'Uso dell'AI Generativa

Per la realizzazione di questo progetto è stato fatto utilizzo di Intelligenza Artificiale Generativa negli ambiti sotto elencati:

- Miglioramento estetico dell'applicativo modificando il codice CSS
- Miglioramento estetico/spaziale dei diagrammi Mermaid presenti nella documentazione del Repository
- Miglioramento dei commenti Inline in determinati file
- Rifattorizzazione di codice Javascript/React per creare moduli più snelli e meno ripetitivi
- Affiancamento a documentazione e forum online per la risoluzione di problemi sorti durante le fasi di testing e debug
- Affiancamento a motori di ricerca per comprendere meglio alcuni concetti teorici prima di essere messi in pratica

I sistemi utilizzati sono stato Gemini Online e Antigravity, entrambi di proprietà di Google.

Le risposte e le modifiche che mi sono state fornite sono state buone ma non sempre del tutto soddisfacenti.

In molti casi hanno reso evidente due di quelli che a mio parere sono i più grandi problemi di questi sistemi: le allucinazioni temporali e il mancato controllo di qualità nelle risposte.

Finché si tratta di chiedere dati statici sono strumenti molto potenti, ma quando si cerca di ottenere risposte riguardo programmi a linea di comando oppure sui linguaggi di programmazione, è molto probabile che le risposte date non siano coerenti temporalmente, e in particolare potrebbero essere inerenti a vecchissime versioni di tali sistemi. La documentazione aggiornata resta dunque la miglior opzione.

Allo stesso modo spesso le risposte date da questi strumenti tendono a preferire l'approccio a "toppe" veloci sugli errori rispetto ad un'implementazione logica.

Per questo motivo resto dell'idea che il modo migliore per adoperare questi strumenti sia quello di avere a monte la consapevolezza dei concetti su cui si ha intenzione di lavorare.

Capitolo 6: Aspetti Implementativi Rilevanti

6.1 Bootstrapping dinamico dei secret in avvio

- Essendo un progetto basato sul mettere in sicurezza un'infrastruttura, volevo evitare di salvare password amministrative o secret dei client OAuth2 in chiaro all'interno dei manifesti statici committati su GitHub
- **Soluzione:** È stato creato un container helper temporaneo ed effimero (quiz-keycloak-setup) che automatizza la configurazione iniziale:
 1. Attende l'avvio completo di Keycloak tramite un ciclo che interroga la CLI ufficiale kcadm.sh.
 2. Effettua l'autenticazione come amministratore temporaneo.
 3. Estrae dinamicamente l'UUID univoco generato da Keycloak per il client oauth2-proxy-client.
 4. Aggiorna il secret del client nel database inserendo il valore sicuro letto dalla variabile d'ambiente del file .env.
 5. Termina con exit code 0, sbloccando l'avvio di oauth2-proxy

6.2 Gestione del Single Log-Out (SLO)

- Il logout standard di oauth2-proxy cancellava solo il cookie di sessione locale, lasciando attiva la sessione SSO su Keycloak. Di conseguenza, provando ad accedere di nuovo, l'utente veniva ri-autenticato automaticamente.
- **Soluzione:** è stato Configurato l'URL di logout di background in oauth2-proxy iniettando la chiave id_token in modo che Keycloak possa convalidarla e distruggere la sessione SSO globale reindirizzando l'utente direttamente alla Landing Page.

6.3 Gestione della compatibilità Dual-Environment

- **Flessibilità di Deploy:** Il sistema è stato progettato per essere totalmente portabile ed eseguire senza alcuna modifica al codice sorgente sia in **Ambiente Locale di Sviluppo** (<http://localhost>) sia in **Ambiente di Produzione** (<https://domain>)

Capitolo 7: Ambiente di produzione

A questo punto, quello che rimaneva da fare era mettere in produzione la piattaforma sul web.

Per fare ciò erano disponibili diverse opzioni, ma alla fine ho deciso di utilizzare il mio server domestico su cui è installato Proxmox.

Esso è un hypervisor di tipo bare-metal che permette di effettuare il deploy di applicativi o di veri e propri sistemi operativi mediante Linux Container (LXC), nei quali il kernel resta condiviso mentre gli userspace sono dedicati, e Macchine Virtuali (VM), dove anche il kernel viene virtualizzato.

Un altro obiettivo che mi ero posto era quello di non dover aprire alcuna porta sul router, in modo da mantenere la mia rete domestica il più sicura possibile; di conseguenza, ho deciso di optare per un tunneling mediante la Content Delivery Network (CDN) di Cloudflare.

L'idea di base è molto semplice: il dispositivo locale comunica esclusivamente con Cloudflare ed il client finale si connette all'IP pubblico di Cloudflare; in questo modo i due attori vengono messi in comunicazione in totale sicurezza.

Per ospitare l'applicazione ho creato un container LXC dedicato con 2 vCPU, 4 GB di RAM e 20 GB di archiviazione, all'interno del quale ho installato il motore Docker.

All'LXC è stato assegnato un IP locale privato, che è stato poi mappato all'interno del mio reverse proxy locale Caddy per instradare il traffico al nuovo stack.

Per quanto riguarda la gestione dei certificati HTTPS e la cifratura delle connessioni web, se ne è occupato direttamente Cloudflare tramite la procedura di verifica DNS-01 Challenge.

Come accennato prima, un punto di forza fondamentale di questo setup è stato la portabilità dell'infrastruttura, infatti grazie alle variabili d'ambiente `${SCHEME}` e `${DOMAIN}`, lo stesso identico stack Docker Compose può essere testato in locale e distribuito in produzione.

Capitolo 8: Schema complessivo del sistema

